ACTL3142 Tutorial

Week 8: Moving Beyond Linearity

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T1 2026

Recap

From Week 7 (CV & Regularisation):

- **Ridge/Lasso**: add a **penalty** on coefficient size to trade a little bias for less variance.
- **Cross-validation**: pick the tuning parameter λ by estimating out-of-sample error.

New direction: so far the model has been **linear in** x. This week we **relax linearity**: transform the predictor before fitting so we can bend the curve to the data.

Both ideas above come back today: smoothing splines penalise **curvature** instead of coefficient size, and we again tune λ by CV.

One predictor first, many at the end

For most of today we work with a **single predictor** x, building a flexible curve $f(x)$. At the very end, **GAMs** stitch one such curve per predictor together to handle many predictors at once.

Today: **Splines** (regression, smoothing), **local regression**, and **GAMs**.

The Basis Function Idea

Every method this week fits the same type of model:

$$y_i = \beta_0 + \sum_{k=1}^{K} \beta_k \, b_k(x_i) + \varepsilon_i$$

where $b_1(x), b_2(x), \ldots, b_K(x)$ are **basis functions**: fixed, known transformations of x.

The model is still **linear in** β, so all our OLS theory applies. The "nonlinearity" comes entirely from the choice of basis functions.

Linear in what?

The model is non-linear in x but linear in the parameters $\beta_0, \beta_1, \ldots, \beta_K$. We can still use `lm()`, compute standard errors, do hypothesis tests, etc.

Polynomial regression, step functions **and splines** *are all just different choices of the* b_k*. (Splines use a* **truncated power basis**, *shown shortly.) The question is: which transformation captures the pattern best?*

Polynomial Regression

The simplest extension: add powers of x as extra predictors.

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \varepsilon_i$$

Basis functions: $b_j(x) = x^j$ for $j = 1, \ldots, d$. Degree d is a hyperparameter (typically 2–4, chosen via CV).

In R: `lm(y ~ poly(x, d))` uses **orthogonal polynomials** for numerical stability (the raw monomials $x, x^2, \ldots$ become highly correlated at high d).

Pros: simple, just extends `lm()`. Inference is straightforward.

Cons: global structure. Every data point influences the fit everywhere, and the curve can behave erratically at the boundaries for large d.

`poly()` uses a rotated basis spanning the same space as raw monomials. Predictions $\hat{y}$ are identical, but $X^\top X$ is well-conditioned for stable computation.

Step Functions

The opposite extreme: assume the relationship is **constant within bins**.

Break the range of x into $K+1$ regions using cutpoints $c_1 < c_2 < \cdots < c_K$:

$$y_i = \beta_0 + \beta_1 I(c_1 \leq x_i < c_2) + \cdots + \beta_K I(c_K \leq x_i) + \varepsilon_i$$

Basis functions: $b_j(x) = I(c_j \leq x < c_{j+1})$ (indicator functions).

In R: `lm(y ~ cut(x, breaks = K))`

Pros: fully **local** (each bin is independent of the others).

Cons: discontinuous at the cutpoints. Ignores ordering within each bin.

Polynomials vs step functions: two extremes

Polynomials are smooth but global (one function everywhere). Step functions are local but discontinuous. Splines combine the best of both.

From Piecewise Polynomials to Splines

Idea: fit separate polynomials in different regions, but require them to connect smoothly at the boundaries (**knots**).

A **spline** of degree d with knots $\xi_1, \ldots, \xi_K$:

- A piecewise polynomial of degree d in each interval between knots
- Continuous up to the $(d-1)$th derivative at each knot

Cubic spline ($d = 3$) is the standard choice:

- Continuous function, continuous 1st and 2nd derivatives at each knot
- The lowest degree where the knots are **invisible to the eye**

In R: `lm(y ~ bs(x, knots = c(...), degree = 3))` from the `splines` package.

Why stop derivative continuity at $d-1$? Matching the dth derivative too would force the pieces to be the **same** *polynomial (no bend at the knot). Stopping one short is the most smoothness we can ask for while still letting the curve change shape.*

Splines Are Still a Basis Expansion

Yes, splines have basis functions too. A cubic spline uses the **truncated power basis**:

$$g(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \theta_k \, (x - \xi_k)_+^3$$

where $(x - \xi_k)_+^3 = (x - \xi_k)^3$ if $x \geq \xi_k$, and 0 otherwise. So it is just `lm()` on these columns.

Why this enforces the spline constraints: at the knot ξ_k, the term $(x - \xi_k)_+^3$ and its 1st and 2nd derivatives are all 0:

$$(x - \xi_k)_+^3, \quad 3(x - \xi_k)_+^2, \quad 6(x - \xi_k)_+ \longrightarrow 0 \text{ at } x = \xi_k.$$

Only the 3rd derivative jumps (from 0 to 6). So adding this term lets the cubic **bend** at ξ_k while keeping g, g', g'' continuous.

Each knot = one basis function = one df

3 global terms (x, x^2, x^3) + one truncated term per knot. Every knot buys exactly one extra parameter of flexibility, at no cost to smoothness.

Natural Splines and Choosing Knots

Problem: cubic splines can behave erratically at the **boundaries** (extrapolation beyond the outermost knots).

Natural spline: a cubic spline forced to be **linear** beyond the boundary knots. Linearity at each end kills the quadratic and cubic terms there: 2 constraints per boundary, **4 in total**.

So its df drops from $K + 4$ to $\mathbf{K}$: none of the df budget is wasted on wild tail behaviour where data is sparse, leaving more for the interior.

In R: `ns(x, df = ...)` or `ns(x, knots = c(...))` from the `splines` package.

Choosing knots: place at **quantiles** of x (uniform in data density). Select the number of knots via CV.

Regression splines vs smoothing splines

Regression splines require you to choose the number *and* location of knots. Smoothing splines avoid this by placing a knot at every data point and penalising roughness instead.

Degrees of Freedom: A Unifying View

Degrees of freedom (df): total free parameters in the fitted curve, **counting the intercept**. Higher df = more flexible.

Method	**Controlled by**	**Total df**
Polynomial	Degree d	$d+1$
Step function	Cuts K	$K+1$
Cubic spline	Knots K	$K+4$
Natural spline	Knots K	K
Smoothing spline	λ	df_λ (non-integer)

Cubic spline: $4(K+1)$ piecewise-cubic parameters $-\ 3K$ continuity constraints $= K+4$. Natural spline removes 4 more (linear tails) $\rightarrow K$.

Watch the intercept: why you also see "$K+3$"

`bs()`, `ns()`, `poly()` return only the **basis columns** (one fewer), since `lm()` adds its own intercept. A cubic spline with K knots gives $K+3$ columns $\rightarrow K+4$ total df.

Smoothing Splines

Now g is a whole **function** (a curve), not a fixed set of coefficients. We search over **all** smooth (twice-differentiable) functions for the one minimising:

$$\min_{g} \quad \frac{1}{n}\sum_{i=1}^{n}(y_i - g(x_i))^2 \; + \; \lambda \int g''(x)^2 \, dx$$

First term: fit the data. Second term: penalise **roughness** ($g'' = 0$ is a straight line, so $\int g''(x)^2 \, dx$ sums total wiggliness). λ controls the tradeoff: $\lambda \to 0$ interpolates every point, $\lambda \to \infty$ gives the least-squares line.

Aren't there infinitely many g that fit?

Without the penalty, **any** curve through all the points gives RSS $= 0$: infinitely many minimisers, all overfit. The roughness penalty breaks the tie, leaving a **unique** minimiser for each λ.

The remarkable part: that minimiser is always a **natural cubic spline** with a knot at every distinct x_i. The spline form **falls out** of the penalty, not an assumption. *(Same spirit as ridge: penalise*

Effective Degrees of Freedom

A knot at every x_i sounds like n parameters, but the penalty **shrinks** them, so the real flexibility is much lower. We measure it with **effective df**.

A smoothing spline is a **linear smoother**: the fitted values are $\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$, where $\mathbf{S}_\lambda$ depends on λ and the x_i, **not** on $\mathbf{y}$. Define

$$\mathrm{df}_\lambda = \mathrm{trace}(\mathbf{S}_\lambda),$$

exactly mirroring OLS, where df = trace of the hat matrix $\mathbf{H}$ = number of parameters.

- Smaller $\lambda \to$ more flexible $\to$ higher df_λ.
- **Non-integer** because the coefficients are shrunk, not free. Ranges from $\mathrm{df}_\lambda = 2$ (line) to $\mathrm{df}_\lambda = n$ (interpolation).

In R: `smooth.spline(x, y, cv = TRUE)` picks λ (hence df) by LOOCV in a single pass. On the Luxembourg mortality data this gave $\mathrm{df}_\lambda \approx 5.08$.

Bonus vs regression splines: no knots or degree to choose, just λ.

Local Regression

Idea: to predict at a target point x_0, fit a **weighted least squares** line using only nearby data.

Algorithm:

1. Gather the fraction $s = k/n$ of training points closest to x_0.
2. Weight them: $K_{i0} > 0$, larger nearer x_0; points outside the k nearest get $K_{i0} = 0$.
3. Fit weighted least squares: $\min_\beta \sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2$.
4. Predict: $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

The sum runs to n, but only k points matter

Step 3 sums over **all** n points, but the far ones have weight $K_{i0} = 0$ and drop out, so only the k neighbours contribute. The weights do the selecting.

Span s tunes flexibility: small s = wiggly, large s = smooth. In R: `loess(y ~ x, span = s)`.

No global equation: it is **memory-based**, *refitting at each* x_0. *Fine for 1–4 predictors; beyond that too few neighbours fall nearby (the "curse of dimensionality"), which GAMs get around.*

Generalised Additive Models (GAMs)

Everything so far was **one predictor**. A GAM is just **one flexible curve per predictor, added up**, the multivariate generalisation:

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i$$

Each f_j can be any method: smoothing spline, natural spline, polynomial, or just linear (mix and match per predictor). Fitted via **backfitting**: hold all curves but f_j fixed and fit f_j to the **partial residuals** $r_i = y_i - \hat{\beta}_0 - \sum_{k \neq j} \hat{f}_k(x_{ik})$, then cycle through all j until nothing changes. Just one curve at a time.

In R: `gam(y ~ s(x1) + s(x2) + x3)` from `mgcv`. `s()` fits a smoothing spline; `x3` enters linearly.

GAMs vs trees

Trees capture **interactions** automatically but are harder to interpret per-variable. GAMs assume **additivity** (no interactions unless explicitly added) but give clear partial effect plots.

GAMs extend to classification:

$\log\left(\frac{p(\mathbf{x})}{\cdots}\right) = \beta_0 + f_1(x_1) + \cdots + f_p(x_p)$